Fake News Detection Using a Feature-Based Optimised Multi-Class SVM: PCA, Firefly Search and a PARTIALLY TRUE Class

Neela Mojes
(23K91A05H1)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Noor Saniya
(23K91A05H4)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Malkajgiri Akshitha
(23K91A05E2)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Maravajhala Radha Sri Aashritha
(23K91A05E7)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

K Rajasekhar Rao
(guide)
[DESIGNATION], Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

***Abstract*---**Fabricated news spreads faster than fact-checks, and many detectors rely on heavy neural models. This paper builds and measures a light alternative: TF-IDF and 25 stylometric features are reduced by principal component analysis (PCA), and the Firefly algorithm chooses the PCA components and the parameters of a multi-class RBF support vector machine (MSVM) that labels a statement REAL, PARTIALLY TRUE or FAKE. The pipeline is packaged as a web application with single and batch checking, analytics and retraining. On the LIAR2 fact-checking dataset, trained on 3,000 statements and scored on 2,296 unseen statements over three seeds, it reaches macro-F1 0.429 ± 0.010 and accuracy 0.494 ± 0.008. A default SVM on the same features, an equal-budget random search and the Firefly search are within noise of each other, and a logistic regression on plain TF-IDF trained in under a second reaches 0.500. The optimiser therefore did not improve accuracy and cost about 504 s of training. PARTIALLY TRUE is the hardest class, and the displayed probabilities disagree with the verdict on about half of the statements.

***Index Terms*---**Fake news detection, feature selection, Firefly algorithm, LIAR2, multi-class SVM, principal component analysis, TF-IDF.

I. INTRODUCTION

Automatic fake news detection is an active research area [1]. False stories reach large audiences before any fact-check is published, and they travel farther and faster than true ones [2]. Manual fact-checking cannot match the volume of online publication, so text classifiers are used to flag suspicious items [3]. Text can be classified from word statistics and writing style [4], [5], but the usual choices are costly in different ways: neural and transformer detectors need GPUs and large corpora [1], [6], while classical pipelines on high-dimensional TF-IDF vectors are slow to tune and easy to overfit.

Most detectors also answer a binary question. Fact-checkers, however, grade claims on a scale, and language differs between false, half-true and true claims [7]; a claim that mixes accurate figures with an exaggerated conclusion is neither real nor fake. Benchmark studies show that such graded data, for example LIAR-PLUS, are the hardest for nearly all models even after the labels are collapsed to two classes [8]. The gap this work addresses is a computationally light pipeline in which feature reduction and metaheuristic tuning aim to deliver useful accuracy at low cost, with an explicit PARTIALLY TRUE class, and an honest measurement of whether it does.

The contributions are: (1) an end-to-end pipeline of TF-IDF and stylometric features, PCA, a Firefly search over the retained components and the SVM parameters, and an RBF multi-class SVM with balanced class weights; (2) a web application that wraps it with single and batch detection, model analytics and retraining; (3) a reproducible benchmark on LIAR2 against mask-only, equal-budget random-search and linear baselines; and (4) findings that are stated as found, including that the optimiser gave no gain beyond noise. Section II reviews related work, Section III the method, Section IV the results and Section V concludes.

A. System Hardware

Every training run and measurement used one desktop computer: Intel Core i5-12500 (6 cores, 12 threads, 3.0 GHz), 15.7 GB RAM, Windows 11 Pro, no GPU used. Training and prediction ran on the CPU only.

B. Software Requirements

Python 3.11.9, scikit-learn 1.9.1, NumPy 2.4.6, SciPy 1.17.1, pandas 3.0.6, FastAPI 0.159.1, uvicorn 0.54.0, Node 24.19.0, React 18.3, Vite 8, TypeScript 5.9, Tailwind CSS 4. The model code uses scikit-learn [9]; the web application is a FastAPI service with a React interface.

II. LITERATURE SURVEY

Base paper. Saad et al. [1] detect Arabic fake news. They compile 7,474 articles from trusted outlets, unverified online content labelled by the authors (Fleiss' kappa 0.80 on a 500-sample subset) and benchmark data partly translated from LIAR and WELFake, and split them 80/20. A neural network on CAMeLBERT contextual embeddings is compared with AraBERT, AraELECTRA and MARBERTv2 embeddings and with resampling (oversampling, undersampling, SMOTE) against class weighting in the loss [10]. Its data come about 45 % from trusted outlets, 30 % from unverified online content and 25 % from benchmark sets; the classifier is a dense layer of 256 units with dropout 0.2, trained with Adam (learning rate 5e-4, batch 16, 10 epochs) on

embeddings of at most 128 tokens. The TF-IDF plus neural-network baseline reaches 90.97 % accuracy; CAMeLBERT with class weighting reaches 95.52 % accuracy, 96.19 % F1, 95.48 % precision and 96.90 % recall on a Tesla T4 GPU, and LIME and SHAP explain the decisions. The task is binary and Arabic, and the authors leave cross-domain generalisation and real-time use as future work. This project keeps three ideas: weight classes instead of resampling, report several metrics, and compare against baselines; it differs in features (TF-IDF, stylometry, PCA), classifier (SVM), a metaheuristic, three classes, English text and CPU-only compute.

Related work. Dataset and benchmark studies: LIAR [11] and its extension LIAR2 [12] supply short fact-checked statements with six truthfulness labels, and an experimental comparison of 12 approaches on 10 English datasets [8] finds LIAR-PLUS the hardest (F1 0.56 for logistic regression, 0.57 for SVM, binary labels). Feature-based detection: n-gram TF-IDF with a linear SVM is a strong classical choice [4], style features separate hyperpartisan from mainstream news but are a weak proxy for truth [5], and language cues differ across degrees of falsehood [7]. Neural models: transformer embeddings dominate recent results [1], and a Transformer with a BiGRU and Bayesian optimisation reports 99.73 % test accuracy on its own corpus [6]. Optimisation: the Firefly algorithm [13] and evolutionary feature selection [14] shrink feature sets for classification [15], and multi-class SVMs are usually built from binary ones [16].

TABLE I
RELATED WORK: APPROACH, DATA, REPORTED RESULT, LIMITATION

Work	Approach	Data	Reported result	Limitation
Saad [1] (base)	CAMeLBERT + NN, class weights	7,474 Arabic articles, binary	Acc. 95.52, F1 96.19 %	Arabic, binary, GPU
Dell'Oglio [8]	12 approaches compared	10 English sets, binary	LIAR-PLUS F1 0.56-0.57 (LR, SVM)	No middle class
Huang [6]	TF-IDF, Transformer, BiGRU, Bayesian opt.	News corpus, binary	99.73 % test acc.	Easy own data
Xu [12]	Fuzzy deep hybrid net; LIAR2	18,369 / 2,297 / 2,296	Benchmark used here	Deep model
Ahmed [4]	N-gram TF-IDF, linear SVM	News corpora, binary	Linear SVM strong	Binary
Potthast [5]	Style features	Fact-checked news	Detects hyperpartisan style	Weak truth proxy
Rashkin [7]	Language of falsehood levels	PolitiFact, news	Cues differ by level	Analysis
Zhang [15]	Firefly feature selection	Benchmark sets	Smaller feature sets	Search cost

Table I shows the gap: transformer detectors report high binary accuracy on their own data, while the cheap, graded and explicit three-class setting is rarely measured. The reported figures come from different data and metrics and are not ranked here.

III. METHODOLOGY

Architecture. Fig. 1 shows the system: a React interface with six screens, a FastAPI service with eight routes (health, model information, samples, single prediction, batch preview and run, retrain and job status) and a pure-Python library that the service, a command-line tool and a Streamlit app all share. The fitted objects are saved once as a joblib bundle and held in memory behind a lock; retraining runs in a background thread and swaps the model in atomically. There is no database.

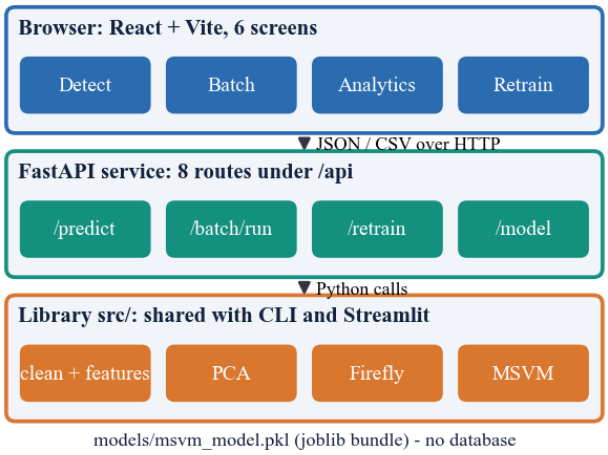


Fig. 1. System architecture: browser, FastAPI service, shared library and the saved model bundle (full diagram: docs/diagrams/01).

Pipeline. Fig. 2 shows the two paths. A text is lower-cased, URLs, mentions and non-letters are removed, stop words and tokens of two characters or fewer are dropped, and TF-IDF [17] with unigrams and bigrams (sublinear term frequency, minimum document frequency 2, at most 4,000 terms) is computed. 25 stylometric features are appended: length and sentence statistics, capital letters, exclamation and question marks, digits, URLs, first-person and superlative rates, and lexicon ratios of sensational and credibility words. The 4,025 columns are standardised and projected by PCA [18], which keeps components up to 95 % of the variance but at most 300.

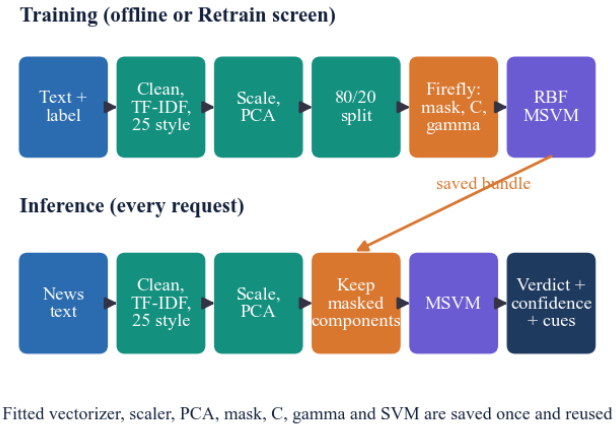


Fig. 2. Training path (top) and inference path (bottom) of the pipeline.

Features and projection. The stylometric block is deliberately cheap: 25 numbers per text that need no language model. PCA [18] finds the orthonormal directions

of largest variance of the standardised feature matrix X; with eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots$ of its covariance matrix, the first c components are kept, where c is the smallest number with

$$(\lambda_1 + \ldots + \lambda_c) / (\lambda_1 + \ldots + \lambda_n) \geq 0.95, \quad c \leq 300. \qquad (1)$$

The cap bounds the cost of the search; on short statements it binds, as Section IV shows. Standardising every TF-IDF column before PCA gives rare terms the same weight as common ones, a design choice that is revisited in the discussion.

Data. LIAR2 [12] has 18,369 training, 2,297 validation and 2,296 test statements of about 18 words, each with one of six PolitiFact labels (licence Apache-2.0). Only the statement text is used; speaker, justification and count columns would leak the label. The project mapping is true and mostly-true to REAL, half-true to PARTIALLY TRUE, and barely-true, false and pants-on-fire to FAKE, giving 602 REAL, 371 PARTIALLY TRUE and 1,323 FAKE test statements; always answering FAKE scores 0.576 accuracy but only 0.244 macro-F1. Because the Firefly search needs hundreds of SVM fits, each run trains on a stratified random subsample of 3,000 statements, of which 80 % fit the classifier; seeds 7, 42, 2024 change the subsample and the search.

Firefly search. Each of 10 fireflies encodes a logit for every PCA component and log10 C in [-1, 3] and log10 gamma in [-4, 1]. A component is kept when its sigmoid exceeds 0.5. The brightness of a firefly is the three-fold cross-validated macro-F1 of a class-weighted RBF SVM on the kept components, minus a size penalty,

$$f(x) = F1_{cv}(m, C, \gamma) - 0.02 \cdot k / n, \qquad (2)$$

where k components of n are kept. Following Yang [13], a firefly i moves towards each brighter firefly j with attractiveness $\beta = \beta_0 \exp(-\gamma_0 r^2/d)$ and a decaying random step,

$$x_i \leftarrow x_i + \beta (x_j - x_i) + \alpha_t \varepsilon, \quad \alpha_t = 0.2 \cdot 0.97^{\wedge}t. \qquad (3)$$

The dimmest firefly is reset near the best one each generation. After 12 generations (about 550 fitness evaluations) the best mask, C and gamma train the final classifier: an RBF SVM [19] with balanced class weights, one-vs-rest decision shape and Platt-scaled probabilities [20]; the three-class problem is handled internally by pairwise coupling [16]. Class weights, as in the base paper [1], counter the imbalance [10].

Classifier. For a masked component vector z, the RBF kernel is $K(z, z') = \exp(-\gamma \|z - z'\|^2)$, and the SVM [19] solves for each class k a one-vs-rest problem with penalty $C \cdot w_k$, where the balanced weight is $w_k = N / (3\ N_k)$ for N_k training statements in class k; rarer classes therefore cost more to misclassify. The label is the class with the largest decision value, and Platt scaling [20] maps decision values to probabilities in a separate cross-validated fit. This is why the label and the probabilities can disagree.

Cost of the search. Every fitness evaluation fits three SVMs on about 1,600 statements, so a search of roughly 550 evaluations means over 1,600 fits. This, not the final fit (about a second), is what takes minutes, and it grows with the number of fireflies and generations, which the Retrain screen exposes as sliders (4-20 fireflies, 3-30 generations, 1,000-8,000 TF-IDF terms).

Application. The Detect screen takes a headline and text (at least five words), returns the verdict, confidence, probabilities and lists of sensational and credibility cue words found in the text. Batch check previews an uploaded CSV, lets the user pick the text column (limits: 25 MB, 20,000 rows) and returns a verdict per row, the class distribution and a results file. Analytics reads the loaded model's statistics, and Retrain accepts a labelled CSV (REAL, PARTIALLY TRUE, FAKE or 0-2, or LIAR names; at least 30 rows), runs the fit in the background and replaces the model without a restart. Invalid input returns 400, 413 or 422 with a message.

Protocol and baselines. Models are scored on the full official test split; macro-F1 (the mean of the three class F1 values) is the main metric because accuracy rewards the majority class. Comparisons use the same features and split: the product's baseline (default RBF SVM on all PCA components); a mask-only variant (Firefly mask, default C and gamma); random search with the same number of evaluations and the same fitness; and linear SVM, logistic regression and complement naive Bayes on plain TF-IDF, on the 3,000-statement subsample and on all 18,369 statements. The shipped demonstration model, trained on 750 synthetic template sentences, is also scored on LIAR2.

Design rationale. TF-IDF with bigrams captures topical and phrasal cues at a cost of milliseconds; the stylometric block adds signals that vocabulary misses, such as shouting, urgency and numeric density. PCA was chosen because a 4,025-dimensional sparse space with a few thousand statements invites overfitting; an RBF SVM copes with the remaining dense space and needs only two parameters. Searching the mask together with C and gamma lets the optimiser trade feature count against accuracy through the size penalty, and a metaheuristic is used because the fitness, a cross-validated score of a discrete mask and two continuous values, has no gradient. Macro-F1 is preferred to accuracy because the classes are unbalanced and the middle class is the one of interest.

Reproducibility. Every random choice (subsample, split, search, Platt folds) is seeded; one script, run_experiments.py, trains each seed, scores the validation and test splits and writes a JSON file per seed, and the aggregate numbers in this paper are read from those files. LIAR2 is downloaded from its public repository and is not redistributed with the code.

Verification. 83 automated checks pass: 16 end-to-end API checks, 34 library checks (cleaning, features, label mapping, Firefly decoding, save and load) and 33 API edge cases (word-count rule, oversized or malformed CSV, parameter ranges, non-English text). The interface was exercised in a browser and the screens of Section IV are captures of it. The server needs 3.7 ms on average per article and scores 2,296 statements in 1.2 s (idle machine, local requests).

IV. RESULTS

Test conditions. All scores are on the 2,296 official LIAR2 test statements, never used for fitting, for three seeds (mean ± standard deviation), on the CPU described in Section I. Table II lists accuracy, macro-F1 and training time. The proposed pipeline reaches macro-F1 0.429 ± 0.010 and accuracy 0.494 ± 0.008 (validation split: 0.439 ± 0.010 and 0.501 ± 0.007).

TABLE II
TEST RESULTS ON LIAR2 (3 CLASSES, MEAN OF 3 SEEDS)

Model	Accuracy	Macro-F1	Train time
Always FAKE	0.576	0.244	-
Compl. NB, TF-IDF, all data	0.599	0.481	0.02 s
Linear SVM, TF-IDF, all data	0.575	0.475	0.34 s
Log. regression, TF-IDF, all data	0.573	0.500	0.83 s
Log. regression, TF-IDF, 3,000	0.554 ± 0.010	0.470 ± 0.011	-
PCA + default RBF SVM	0.505 ± 0.011	0.441 ± 0.010	1.20 s (fit only)
PCA + Firefly mask only	0.481 ± 0.015	0.425 ± 0.014	-
PCA + random search (equal budget)	0.494 ± 0.019	0.436 ± 0.009	480 s
PCA + Firefly + MSVM	0.494 ± 0.008	0.429 ± 0.010	504 s
Shipped demo model (synthetic)	0.192	0.155	-

What the numbers say. The three PCA-based variants are within about one standard deviation of each other (Table II): Firefly 0.429 ± 0.010, random search 0.436 ± 0.009, default SVM 0.441 ± 0.010. On the training split the Firefly fitness is slightly higher than the best random-search fitness (0.433 against 0.426), but this does not carry to the test split. Plain TF-IDF linear models are better and far cheaper: logistic regression on all statements scores 0.500 in 0.8 s, against about 504 s for the Firefly pipeline. PCA is capped at 300 components and keeps only 29 % of the variance of the standardised features; Firefly keeps 148 ± 12 of them. The synthetic-data model, whose own hold-out score is 1.000, falls to 0.155 macro-F1 on real statements and answers PARTIALLY TRUE for 2,044 of 2,296.

Hold-out versus test. The Analytics screen reports the score of the model on a 20 % hold-out of its own training subsample (49.5 % accuracy, macro-F1 0.413 ± 0.003); the official test split gives 0.429 ± 0.010, so the in-application figure is a fair estimate here, unlike the 1.000 that the demonstration model shows for its synthetic data.

Data size and simple models. With the same 3,000 statements, logistic regression on TF-IDF scores 0.470 ± 0.011, above the proposed pipeline; trained on all statements it reaches 0.500. More data therefore helps a linear model by 0.031 macro-F1, which suggests that the 3,000-statement budget of the Firefly pipeline, not only its design, limits it. The Firefly pipeline was not run on all statements.

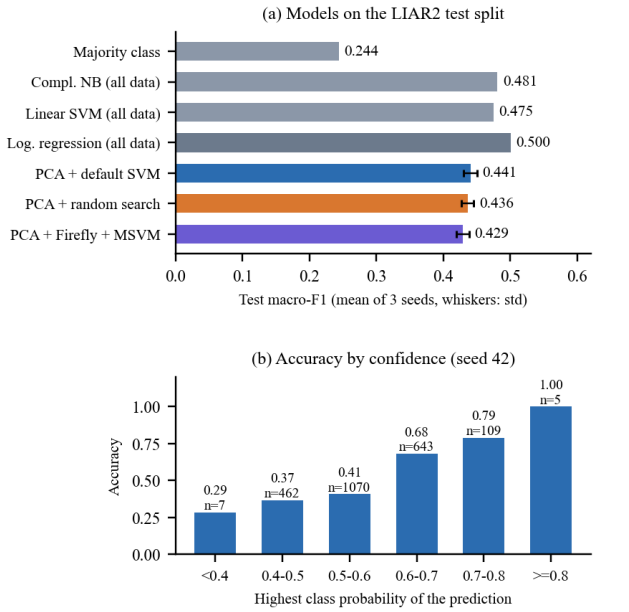


Fig. 3. (a) Test macro-F1 of the proposed pipeline and its comparisons; (b) accuracy of the proposed pipeline by confidence bin.

Per class. FAKE is the easiest class (F1 0.63, precision 0.71), REAL is middling (0.42) and PARTIALLY TRUE is the hardest (0.24, precision 0.21): half-true claims have no distinctive wording and are spread over the other classes. Accuracy falls slowly with statement length, from 0.56 for under 10 words to 0.43 for 30 words or more. Confidence is informative (Fig. 3b): accuracy is 0.37 when the top probability is 0.4-0.5 and 0.68 at 0.6-0.7.

Verdict versus probability. The verdict comes from the SVM decision function and the probabilities from Platt scaling, and they disagree: on 1,120 of 2,296 statements (48.8 %, seed 42) the verdict is not the most probable class, because the probabilities almost always favour FAKE (2,252 statements) and never PARTIALLY TRUE. Fig. 4 shows the Detect screen; for the PARTIALLY TRUE example captured the same way, the verdict was PARTIALLY TRUE at a displayed 47.2 % while the FAKE bar was the tallest, because the interface reports the largest probability as confidence, which can belong to a different class from the verdict. Using the most probable class as the verdict instead would give accuracy 0.578 but macro-F1 0.264.

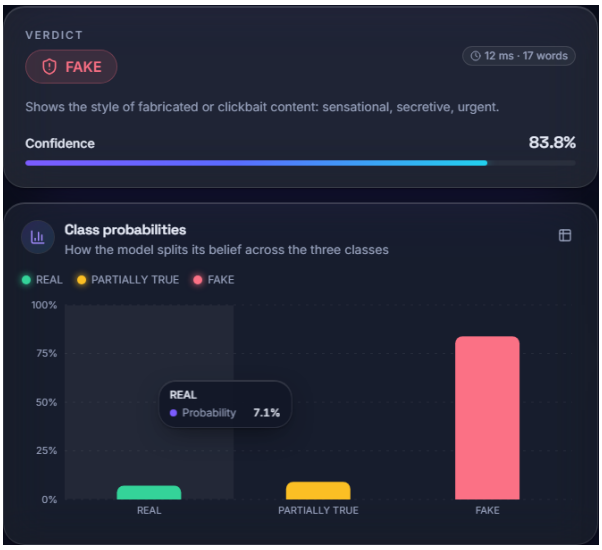


Fig. 4. Output screen (Detect): a real test statement scored FAKE with class probabilities and cue words, from the model trained on LIAR2.

Failure cases. A FAKE-labelled test statement about the signers of the Declaration of Independence is read as REAL with 53.4 % confidence, and a REAL-labelled one about a public figure's degree is read as FAKE with 72.8 % probability: the model sees wording, not facts, and cannot check a claim against evidence. Because it uses text alone, its verdicts are screening aids, not fact-checks.

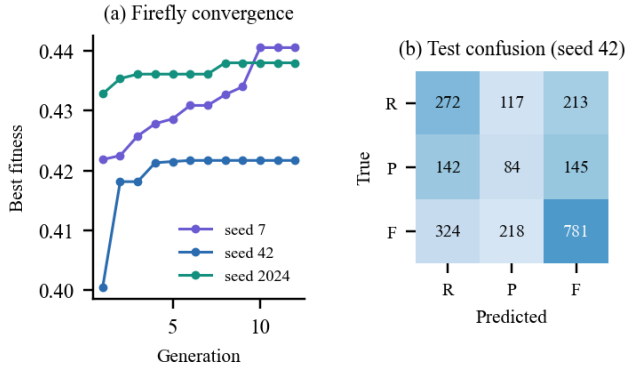


Fig. 5. (a) Best fitness per Firefly generation for each seed; (b) test confusion matrix of seed 42 (R: REAL, P: PARTIALLY TRUE, F: FAKE).

Optimiser behaviour. The search converges within about five generations (Fig. 5a) and then stalls, so most of its 12 generations add nothing. Per seed (2024, 42, 7) the test macro-F1 of Firefly is 0.433, 0.418, 0.436, of the default SVM 0.451, 0.440, 0.431 and of random search 0.439, 0.444, 0.426; the order changes from seed to seed, which is what 'within noise' means here. Plausible reasons, none of which was tested, are that three-fold cross-validation on about 2,400 statements is noisy at the level of ±0.01 F1, that the fitness surface is flat for C and gamma near their useful range, and that a mask over 300 components of a projection that keeps 29 % of the variance has little signal to select. Confusions (Fig. 5b) run mostly between neighbouring grades: 324 FAKE statements read as REAL and 213 REAL read as FAKE in seed 42, and 287 of 371 PARTIALLY TRUE statements fall into the other two classes.

Speed and use. With the machine idle, a single prediction takes 3.7 ms in the server (95th percentile 5.3 ms) and 12.6 ms round trip including HTTP (95th percentile 29.8 ms); a batch of 1,000 statements takes 0.55 s and the whole test split 1.2 s; the service starts in 6 s. The Retrain screen was exercised end to end on a 600-statement CSV with 4 fireflies and 3 generations: it finished in 199 s and reported accuracy 42.5 % and macro-F1 0.411, a reminder that the cost scales with the search settings and that very small training sets give weaker models. All of these figures are for local, sequential requests; concurrency was not tested.

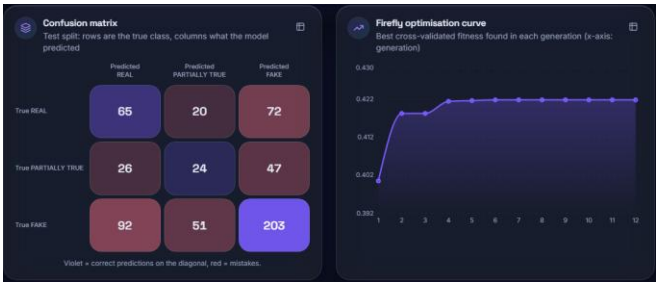


Fig. 6. Output screen (Model analytics): confusion matrix and Firefly curve of the model trained on LIAR2 (hold-out of its own training subsample).

Comparison with the base paper. The base paper's 95.52 % accuracy and 96.19 % F1 [1] are not comparable with these numbers. It classifies Arabic articles into two classes with CAMeLBERT embeddings on a GPU, whereas this project classifies short English statements into three classes with classical features on a CPU, on a dataset where even binary classifiers score F1 of about 0.56-0.57 [8]. The two results show different tasks, not a better or worse method. The project did reproduce the base paper's practice of weighting classes and comparing with baselines.

Threats to validity. The data are political statements from one fact-checking organisation, so the results say nothing about other genres or languages. The three seeds share the same test split, so the standard deviations measure training variation, not test-sample variation; with 2,296 statements a binomial interval on accuracy is about ±2 points. Label mapping is a modelling choice: barely-true is counted as FAKE, as in the project's loader, and a different mapping would change every score. Finally, scores were not tuned on the test split, but the Firefly and random searches shared one fitness function, which favours neither.

Limits. Training used 3,000 of 18,369 statements for compute reasons; three seeds were run and no significance test was made; Firefly's own parameters were not tuned; stemming, listed among the project's preprocessing aims, is not implemented; no other dataset, user study or load test was run; and a CSV label column that holds only the words true and false is parsed as booleans and mislabelled by the loader.

V. CONCLUSION

The system was built and measured: a PCA, Firefly and MSVM pipeline with a PARTIALLY TRUE class, behind a web application, with 83 automated checks passing. On

LIAR2 it reaches macro-F1 0.429 ± 0.010 and accuracy 0.494 ± 0.008, well above always answering FAKE on macro-F1 but below a TF-IDF logistic regression (0.500). Against the objectives: the dataset, preprocessing (without stemming), PCA, Firefly selection, MSVM training and evaluation against an unoptimised baseline were done; the claimed improvement in accuracy and cost was not achieved on this data, since the Firefly search matched random search and the default SVM within noise at far higher training cost.

The main finding is that the optimiser's benefit is not demonstrated, that PARTIALLY TRUE is hard to recognise from wording alone, and that confidence is only partly reliable because verdict and probabilities come from different mechanisms. Future work: calibrate the probabilities on class-weighted data; keep more PCA variance or skip standardising sparse TF-IDF; tune the Firefly parameters and train on all statements; add evidence retrieval; and test on further datasets and with users. Until such work shows a gain, the simpler linear baseline is the better engineering choice on this data, and the application's value lies in its packaging, explanations and honest analytics rather than in the optimiser.

REFERENCES

[1] M. Saad, S. Abdelrazek and I. R. Abdelmaksoud, "An algorithmic system for Arabic fake news detection using neural networks and transformer embeddings with class weighting," Scientific Reports, vol. 16, Art. no. 12226, 2026, doi: 10.1038/s41598-026-45653-4.

[2] S. Vosoughi, D. Roy and S. Aral, "The spread of true and false news online," Science, vol. 359, no. 6380, pp. 1146-1151, Mar. 2018.

[3] K. Shu, A. Sliva, S. Wang, J. Tang and H. Liu, "Fake news detection on social media: A data mining perspective," ACM SIGKDD Explorations Newsletter, vol. 19, no. 1, pp. 22-36, 2017.

[4] H. Ahmed, I. Traore and S. Saad, "Detection of online fake news using N-gram analysis and machine learning techniques," in Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments (ISDDC 2017), Lecture Notes in Computer Science, vol. 10618. Cham, Switzerland: Springer, 2017, pp. 127-138.

[5] M. Potthast, J. Kiesel, K. Reinartz, J. Bevendorff and B. Stein, "A stylometric inquiry into hyperpartisan and fake news," in Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (ACL), Melbourne, Australia, 2018, pp. 231-240.

[6] T. Huang, Z. Xu, P. Yu, J. Yi and X. Xu, "A hybrid transformer model for fake news detection: Leveraging Bayesian optimization and bidirectional recurrent unit," 2025, arXiv:2502.09097.

[7] H. Rashkin, E. Choi, J. Y. Jang, S. Volkova and Y. Choi, "Truth of varying shades: Analyzing language in fake news and political fact-checking," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Copenhagen, Denmark, 2017, pp. 2931-2937.

[8] P. Dell'Oglio, A. Bondielli, F. Marcelloni and L. C. Passaro, "An experimental comparison of the most popular approaches to fake news detection," Information Sciences, 2026, doi: 10.1016/j.ins.2026.123407; arXiv:2603.25501.

[9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[10] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263-1284, Sep. 2009.

[11] W. Y. Wang, "'Liar, liar pants on fire': A new benchmark dataset for fake news detection," in Proc. 55th Annu. Meeting Assoc. Comput. Linguistics (ACL), Vol. 2: Short Papers, Vancouver, Canada, 2017, pp. 422-426, doi: 10.18653/v1/P17-2067.

[12] C. Xu and M.-T. Kechadi, "An enhanced fake news detection system with fuzzy deep learning," IEEE Access, vol. 12, pp. 88006-88021, 2024, doi: 10.1109/ACCESS.2024.3418340.

[13] X.-S. Yang, "Firefly algorithms for multimodal optimization," in Stochastic Algorithms: Foundations and Applications (SAGA 2009), Lecture Notes in Computer Science, vol. 5792. Berlin, Germany: Springer, 2009, pp. 169-178.

[14] B. Xue, M. Zhang, W. N. Browne and X. Yao, "A survey on evolutionary computation approaches to feature selection," IEEE Trans. Evol. Comput., vol. 20, no. 4, pp. 606-626, Aug. 2016.

[15] L. Zhang, K. Mistry, C. P. Lim and S. C. Neoh, "Feature selection using firefly optimization for classification and regression models," Decision Support Systems, vol. 106, pp. 64-85, 2018.

[16] C.-W. Hsu and C.-J. Lin, "A comparison of methods for multiclass support vector machines," IEEE Trans. Neural Netw., vol. 13, no. 2, pp. 415-425, Mar. 2002.

[17] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988.

[18] I. T. Jolliffe and J. Cadima, "Principal component analysis: a review and recent developments," Phil. Trans. R. Soc. A, vol. 374, no. 2065, Art. no. 20150202, 2016.

[19] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273-297, Sep. 1995.

[20] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers. Cambridge, MA, USA: MIT Press, 1999, pp. 61-74.